

GitHub Skills 第 6 节课柴建铭 2023250361021

柴建铭

2026 年 5 月

GitHub Skills 第 6 节课柴建铭 2023250361021

课程名称: Release-based workflow

报告主题: 基于 Release 的版本发布 workflow

学生姓名: 柴建铭

学号: 2023250361021

一、核心知识点梳理

本节课学习基于 Release 的版本发布 workflow。Release 是对项目某个稳定版本的打包和说明，通常会绑定一个 Git tag，用于标记项目历史中的一个具体版本。与普通提交相比，Release 更适合面向用户或团队成员发布可下载、可追踪、可回退的版本。

本节核心知识包括 beta release、release branch、release notes、正式发布和 hotfix。Beta release 通常用于发布测试版本，便于提前验证功能；release branch 可以在临近发布时集中处理稳定性问题；release notes 用于说明本版本新增内容、修复问题和已知限制；hotfix 则是在正式版本发布后发现紧急问题时，对稳定版本进行快速修复并重新发布。

二、学习中的难点

学习中的难点是区分 commit、tag 和 release。Commit 是每一次修改记录，tag 是给某个提交点上版本标记，而 release 是基于 tag 形成的发布说明和资产集合。刚开始容易认为只要代码提交了就是发布，但真正面向用户时，还需要明确版本号、变更说明和下载内容。

另一个难点是理解为什么需要发布分支。对于小项目来说，直接在 main 分支发布似乎更简单；但当项目需要同时开发新功能和维护稳定版本时，release branch 可以把即将发布的版本从日常开发中隔离出来，减少未完成新功能混入正式版本的风险。

三、实践操作与收获

实践过程中，我按照课程步骤创建 beta release，理解预发布版本的含义；随后在 release 分支中补充功能或修复内容，发起 release pull request，把准备发布的修改合并到主线中。在生成 release notes 时，我整理了本次版本的主要变化，使版本说明能够让使用者快速了解更新内容。最后通过正式 release 和 hotfix 的练习，理解了版本发布后仍可能需要维护修复分支。

本次实践让我认识到软件工程不仅包含写代码，还包括版本管理和交付管理。对于课程项目或机器人项目，如果每次实验成果都能用版本号记录，例如 v1.0 完成颜色识别、v1.1 完成舵机控制、v1.2 修复抖动问题，那么后续复盘和展示会更加清晰。Release 工作流程能够让项目成果从“代码堆积”变成“阶段性交付”。

四、学习遇到的困惑与疑问（选做）

目前的疑问是：版本号应该如何制定才合理？例如 v1.0.0、v1.1.0 和 v1.1.1 分别适合什么修改场景？在团队项目中，release notes 应该由开发者编写，还是由项目负责人统一整理？

五、报告小结

通过本节 GitHub Skills 课程学习，我不仅完成了平台提供的实践任务，也进一步理解了软件工程中“版本控制、协作开发、文档规范、流程管理”的实际意义。后续学习中，我会继续按照规范提交内容，保持报告结构完整、表达准确、实践过程具体，并在项目训练中主动使用 GitHub 记录每一次关键修改。